

PAYMENT MANAGEMENT

SHOPSPHERE ONLINE E-COMMERCE DATABASE MANAGEMENT SYSTEM

1.Database Design :

The Payment entity is designed to store and manage payment transaction details for customer orders in the ShopSphere Online E-Commerce Database Management System. It records information such as Payment ID, Order ID, Payment Mode, Payment Status, Payment Date, and Payment Amount. The Payment_ID uniquely identifies each payment transaction and acts as the Primary Key. The Order_ID connects the Payment table with the Orders table and acts as a Foreign Key. The Payment table helps to manage successful and failed transactions and provides useful information for analyzing payment methods and generating payment transaction reports.

1.1 Payment Table :

Attribute	Key Type
Payment_ID	Primary Key
Order_ID	Foreign Key
Payment_Mode	-
Payment_Status	-
Payment_Date	-
Payment_Amount	-

2. Create Payment Table :

```
SQL> CREATE TABLE Payment (  
2     Payment_ID INT PRIMARY KEY,  
3     Order_ID INT NOT NULL,  
4     Payment_Mode VARCHAR2(30) NOT NULL,  
5     Payment_Date DATE NOT NULL,  
6     Payment_Status VARCHAR2(20) NOT NULL,  
7     Payment_Amount NUMBER(10,2) NOT NULL,  
8     CONSTRAINT fk_payment_order  
9     FOREIGN KEY (Order_ID)  
10    REFERENCES Orders(Order_ID)  
11 );
```

Table created.

3. Payment Records :

```
SQL> INSERT INTO Payment  
2 (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)  
3 VALUES  
4 (501, 1001, 'UPI', DATE '2026-09-15', 'Successful', 1499.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Payment  
2 (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)  
3 VALUES  
4 (502, 1002, 'Credit Card', DATE '2026-09-16', 'Successful', 2499.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Payment  
2 (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)  
3 VALUES  
4 (503, 1003, 'Debit Card', DATE '2026-09-17', 'Failed', 899.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Payment  
2 (Payment_ID, Order_ID, Payment_Mode, Payment_Date, Payment_Status, Payment_Amount)  
3 VALUES  
4 (504, 1004, 'Cash on Delivery', DATE '2026-09-18', 'Successful', 3298.00);
```

1 row created.

4. Display Payment Records :

```
SQL> SELECT *
      2 FROM Payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D
501	1001	UPI	15-SEP-26
Successful		1499	
502	1002	Credit Card	16-SEP-26
Successful		2499	
503	1003	Debit Card	17-SEP-26
Failed		899	

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D
504	1004	Cash on Delivery	18-SEP-26
Successful		3298	

5. Successful Transactions :

```
SQL> SELECT *
      2 FROM Payment
      3 WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D
501	1001	UPI	15-SEP-26
Successful		1499	
502	1002	Credit Card	16-SEP-26
Successful		2499	
504	1004	Cash on Delivery	18-SEP-26
Successful		3298	

6. Failed Transactions :

```
SQL> SELECT *  
  2 FROM Payment  
  3 WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D
503	1003	Debit Card	17-SEP-26
Failed		899	

7. Modify Payment Status :

```
SQL> UPDATE Payment  
  2 SET Payment_Status = 'Successful'  
  3 WHERE Payment_ID = 503;
```

1 row updated.

```
SQL> UPDATE Payment  
  2 SET Payment_Status = 'Failed'  
  3 WHERE Payment_ID = 503;
```

1 row updated.

8. ANALYZE PAYMENT METHODS :

8.1 COUNT TRANSACTIONS BY PAYMENT METHOD :

```
SQL> SELECT
  2     Payment_Mode,
  3     COUNT(*) AS Transaction_Count
  4 FROM Payment
  5 GROUP BY Payment_Mode
  6 ORDER BY Payment_Mode;
```

PAYMENT_MODE	TRANSACTION_COUNT
-----	-----
Cash on Delivery	1
Credit Card	1
Debit Card	1
UPI	1

8.2 TOTAL AMOUNT BY PAYMENT METHOD :

```
SQL> SELECT
  2     Payment_Mode,
  3     SUM(Payment_Amount) AS Total_Amount
  4 FROM Payment
  5 GROUP BY Payment_Mode
  6 ORDER BY Payment_Mode;
```

PAYMENT_MODE	TOTAL_AMOUNT
-----	-----
Cash on Delivery	3298
Credit Card	2499
Debit Card	899
UPI	1499

9. PAYMENT TRANSACTION REPORT :

```

SQL> SELECT
  2     p.Payment_ID,
  3     p.Order_ID,
  4     c.Customer_ID,
  5     c.Customer_Name,
  6     p.Payment_Mode,
  7     p.Payment_Date,
  8     p.Payment_Status,
  9     p.Payment_Amount
10 FROM Payment p
11 JOIN Orders o
12     ON p.Order_ID = o.Order_ID
13 JOIN Customer c
14     ON o.Customer_ID = c.Customer_ID
15 ORDER BY p.Payment_ID;

```

```

PAYMENT_ID  ORDER_ID  CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PAYMENT_MODE                PAYMENT_D  PAYMENT_STATUS                PAYMENT_AMOUNT
-----
          501          1001          101
Arun Kumar
UPI                                15-SEP-26  Successful                1499

          502          1002          102
Priya Sharma
Credit Card                    16-SEP-26  Successful                2499

```

```

PAYMENT_ID  ORDER_ID  CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PAYMENT_MODE                PAYMENT_D  PAYMENT_STATUS                PAYMENT_AMOUNT
-----

```

```

-----
          503          1003          103
Rahul Raj
Debit Card                    17-SEP-26  Successful                899

          504          1004          104
Divya Sri

```

```

PAYMENT_ID  ORDER_ID  CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PAYMENT_MODE                PAYMENT_D  PAYMENT_STATUS                PAYMENT_AMOUNT
-----
Cash on Delivery                18-SEP-26  Successful                3298

```

Conclusion:

The Payment Management module of the ShopSphere Online E-Commerce Database Management System successfully stores payment details, manages successful and failed transactions, analyzes payment methods, and generates payment transaction reports.